

Designing a Volunteer Matching System (VMS) with the Finnish Red Cross (FRC)

CS-A1155: Databases for Data Science 2024

COURSE PROJECT - GROUP 12

Authors: Anh Pham, Hoang Tran, Egor Kokko, Marichka Horodyska

1. Introduction

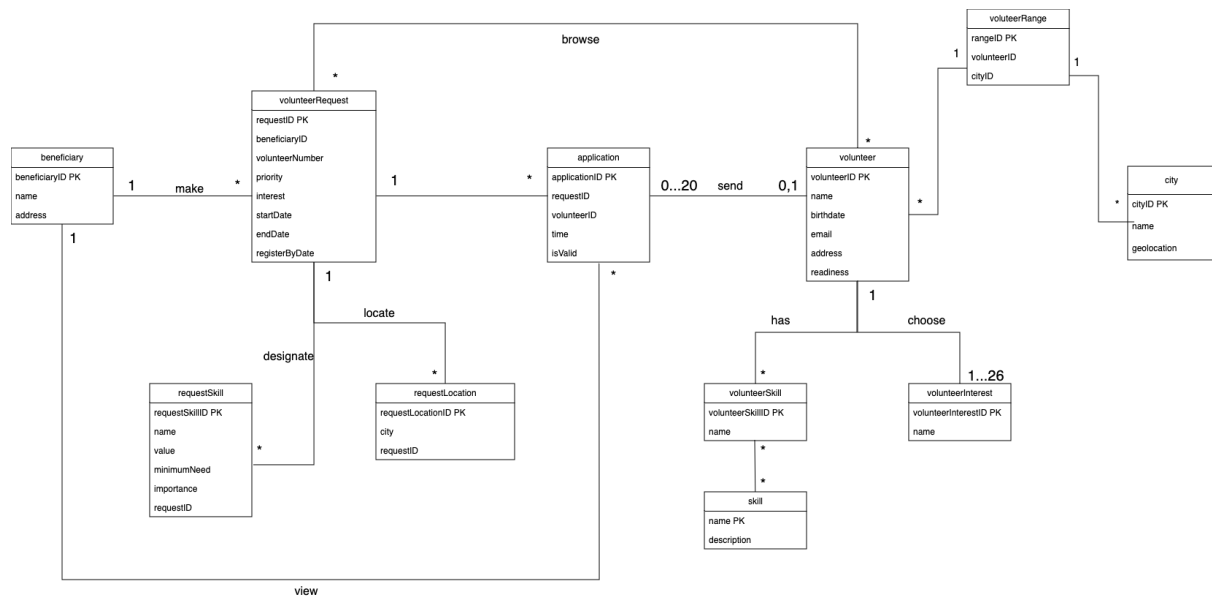
In today's rapidly changing world, the demand for efficient and effective volunteer management has never been more critical. To address this need, we have embarked on a project to design a comprehensive Volunteer Matching System (VMS) in collaboration with the Finnish Red Cross (FRC). This project is a key component of our coursework for the “ Databases for Data Science 2024 “ course at Aalto University, and has been undertaken by Group 12, comprising Anh Pham, Hoang Tran, Egor Kokko, and Marichka Horodyska.

Our project involves seven sections: Unified Modeling Language (UML) diagram and relational model, SQL Implementation, dynamic volunteer matching, volunteer management dashboard, role division, conclusion and appendix.

Part 1 Summary

2. UML diagram and relation model

2.1 UML diagram



We developed a UML (2.1) diagram to represent the structure of our Volunteer Matching System (VMS). This representation provided a clear view of our database design, allowing us to manage volunteer requests and other related activities efficiently.

2.2 Explanations for Each Table and Relation

Table	Purpose	Relationship
Beneficiary	Stores information about beneficiaries.	Linked to VolunteerRequest table through beneficiaryID.

volunteerRequest	Stores requests made by beneficiaries for volunteer help.	Linked to Beneficiary, RequestSkill, RequestLocation, and Application tables.
requestSkill	Stores requests made by beneficiaries for volunteer help.	Linked to VolunteerRequest table through requestID.
requestLocation	Stores locations associated with each volunteer request.	: Linked to City and VolunteerRequest tables through cityID and requestID.
Application	Stores applications submitted by volunteers for requests.	Linked to VolunteerRequest and Volunteer tables through requestID and volunteerID.
volunteer	Stores information about volunteers.	Linked to VolunteerSkill, VolunteerInterest, Application, and VolunteerRange tables.
volunteerSkill	Stores skills that each volunteer has.	Linked to Volunteer and Skill tables through volunteerID and skillID.
volunteerInterest	Stores areas of interest for each volunteer.	Linked to the Volunteer table through volunteerID.

skill	Stores details about skills.	Linked to VolunteerSkill table through skillID.
city	Stores details about cities.	Linked to RequestLocation and VolunteerRange tables through cityID.
volunteerRange	Associates volunteers with cities they can operate in.	Linked to Volunteer and City tables through volunteerID and cityID.

2.3 Relational Data Model

beneficiary (beneficiaryID, name, address)

volunteerRequest (requestID, beneficiaryID, volunteerNumber, priority, interest, startDate, endDate, registerByDate)

requestSkill (requestSkillID, name, value, minimumNeed, importance, requestID)

requestLocation (requestLocationID, city, requestID)

application (applicationID, requestID, volunteerID, time, isValid)

volunteer (volunteerID, name, birthdate, email, address, readiness)

volunteerSkill (volunteerSkillID, skill, volunteerID)

volunteerInterest (volunteerInterestID, name, volunteerID)

skill (name, description)

city (cityID, name, geolocation)

volunteerRange (rangeID, volunteerID, cityID)

browse(requestID,volunteerID)

view(beneficiaryID, applicationID)

The relational data model (2.2) we constructed outlines key entities and their relationships within the VMS. These entities include beneficiaries, volunteer

requests, volunteers, skills, and cities. Each entity is uniquely identified and linked via foreign keys, ensuring data integrity and simplifying complex queries. This model supports **tracking volunteer applications, matching volunteers to requests based on skills and locations, and monitoring volunteer readiness and interests**.

2.4 Assessing the Model

Beneficiary(beneficiaryID, name, address)

VolunteerRequest(requestID, beneficiaryID, volunteerNumber, priority, interest, startDate, endDate, registerByDate)

FK: beneficiaryID references Beneficiary(beneficiaryID)

RequestSkill(requestSkillID, name, value, minimumNeed, importance, requestID)

FK: requestID references VolunteerRequest(requestID)

RequestLocation(requestLocationID, cityID, requestID)

FK: cityID references City(cityID)

FK: requestID references VolunteerRequest(requestID)

Application(applicationID, requestID, volunteerID, time, isValid)

FK: requestID references VolunteerRequest(requestID)

FK: volunteerID references Volunteer(volunteerID)

Volunteer(volunteerID, name, birthdate, email, address, readiness)

VolunteerSkill(volunteerSkillID, skillID, volunteerID)

FK: skillID references Skill(skillID)

FK: volunteerID references Volunteer(volunteerID)

VolunteerInterest(volunteerInterestID, name, volunteerID)

FK: volunteerID references Volunteer(volunteerID)

Skill(skillID, name, description)

City(cityID, name, geolocation)

VolunteerRange(rangeID, volunteerID, cityID)

FK: volunteerID references Volunteer(volunteerID)

FK: cityID references City(cityID)

2.5 Boyce-Codd Normal Form

Beneficiary

FD: beneficiaryID → name, address

beneficiaryID is the candidate key and determinant.

VolunteerRequest

FD: requestID → beneficiaryID, volunteerNumber, priority, interest, startDate, endDate, registerByDate

requestID is the candidate key and determinant.

RequestSkill

FD: requestSkillID → name, value, minimumNeed, importance, requestID

requestID, name → value, minimumNeed, importance (if name is unique for each request)

requestSkillID is the candidate key and determinant.

Ensure that requestSkillID is sufficient to identify each record uniquely.

RequestLocation

FD: requestLocationID → cityID, requestID

requestID, cityID → requestLocationID

requestLocationID is the candidate key and determinant.

Ensure that requestLocationID is sufficient to identify each record uniquely.

Application

FD: applicationID $\rightarrow$ requestID, volunteerID, time, isValid

applicationID is the candidate key and determinant.

Volunteer

FD: volunteerID $\rightarrow$ name, birthdate, email, address, readiness

volunteerID is the candidate key and determinant.

VolunteerSkill

FD: volunteerSkillID $\rightarrow$ skillID, volunteerID

volunteerID, skillID $\rightarrow$ volunteerSkillID (if the combination of volunteerID and skillID is unique)

volunteerSkillID is the candidate key and determinant.

Ensure that volunteerSkillID is sufficient to identify each record uniquely.

VolunteerInterest

FD: volunteerInterestID $\rightarrow$ name, volunteerID

volunteerID, name $\rightarrow$ volunteerInterestID (if name is unique for each volunteer)

volunteerInterestID is the candidate key and determinant.

Ensure that volunteerInterestID is sufficient to identify each record uniquely.

Skill

FD: skillID $\rightarrow$ name, description

skillID is the candidate key and determinant.

City

FD: cityID $\rightarrow$ name, geolocation

cityID is the candidate key and determinant.

VolunteerRange

FD: rangeID $\rightarrow$ volunteerID, cityID

volunteerID, cityID $\rightarrow$ rangeID (if the combination of volunteerID and cityID is unique)

rangeID is the candidate key and determinant.

Ensure that rangeID is sufficient to identify each record uniquely.

Browse

FD: requestID, volunteerID $\rightarrow$ requestID, volunteerID

The left-hand side is a candidate key.

View:

FD: beneficiaryID, applicationID $\rightarrow$ beneficiaryID, applicationID

The left-hand side is a candidate key.

Anomaly Analysis

By achieving BCNF, we minimised redundancy and ensure data integrity. However, we did not analyse anomalies thoroughly in the initial design. Here's an updated analysis:

- Insertion Anomaly: Adding a new volunteer without specifying skills can be problematic. This is addressed by ensuring that volunteerSkill is an association class with volunteerID and skillID as composite keys.
- Update Anomaly: Changing a volunteer's address in one place should reflect in all related records. This is managed by maintaining a single Volunteer table.
- Deletion Anomaly: Deleting a volunteer's record should not remove critical skill data. By treating volunteerSkill as an association class, this ensures skill data integrity.

As a result, our system is now fully prepared to match volunteers to requests effectively.

Part 2 Summary

In Part 2 of our project, we focused on the SQL implementation necessary for the Volunteer Matching System (VMS) to function effectively. This phase was critical for converting our theoretical models into practical, working components that support the operational needs of the Finnish Red Cross (FRC). The scope of this part covered database changes, SQL queries, views, triggers and functions, transactions, analysis, dynamic volunteer matching, and the development of the Volunteer Management Dashboard.

We began by making essential changes to the initial UML diagram and relational model, such as adding cityID to the beneficiary table and changing the interest attribute to title in the volunteerRequest table. These changes were necessary to create a more comprehensive and accurate database structure.

Following the adjustments, we developed a series of SQL queries designed to facilitate various functionalities within the VMS. These queries were fundamental in tasks such as matching volunteer skills to requests, calculating volunteer shortages, retrieving volunteer and beneficiary data, and analysing skill demand. Each query was to address specific needs, thereby enhancing the overall efficiency of volunteer management.

We also created views to provide aggregated data and insights, enabling administrators to make more informed decisions. To maintain data integrity and automate processes, we implemented several triggers and functions. A notable function was a check constraint that validated volunteer IDs when a new volunteer was inserted, ensuring the IDs were accurate and consistent. Additionally, we developed a trigger that dynamically updated the number of volunteers needed for requests whenever there was a change in the minimum skill requirements.

Transactions were designed and executed to ensure the consistency and integrity of volunteer assignments, filtering and matching volunteers based on their skills and location.

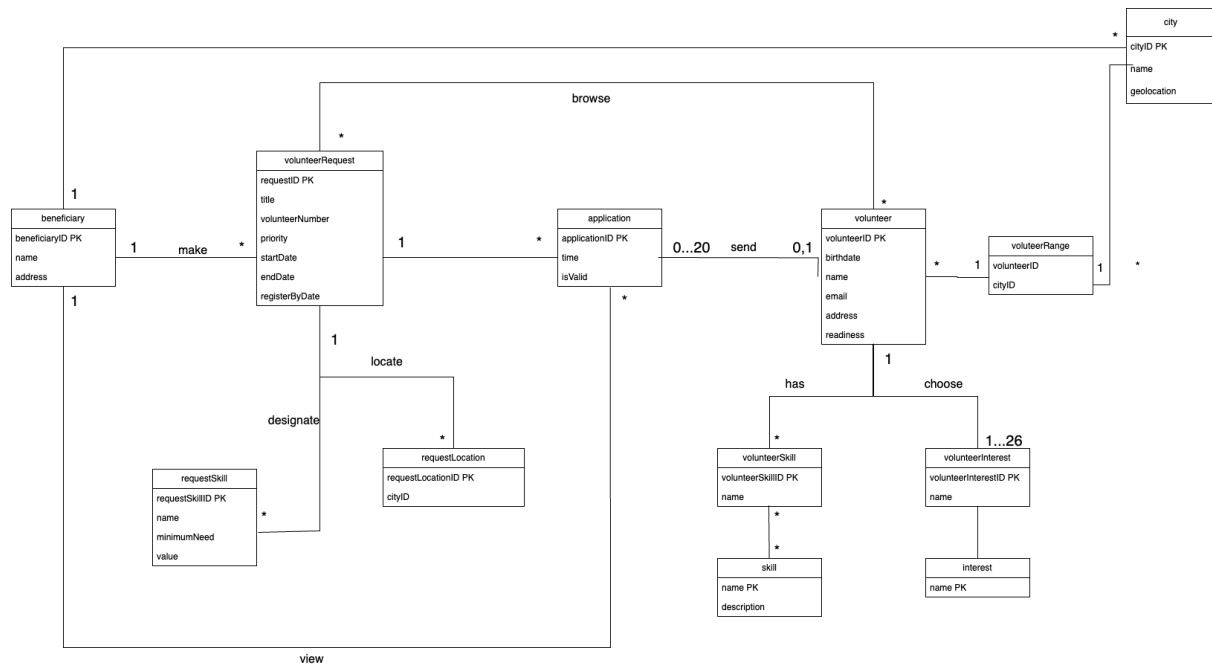
Moreover, we conducted comprehensive analyses to gain a deeper understanding of volunteer distribution and readiness. One of the significant outcomes was the development of a scoring system for matching volunteers to requests. This system took into account various factors such as skill match, travel readiness, and interest alignment, providing a more nuanced approach to volunteer matching.

In the dynamic volunteer matching section, we developed a system that allocates volunteers to requests based on real-time data, ensuring responsiveness during emergencies and crises. The Volunteer Management Dashboard was developed to

provide an interactive and comprehensive view of volunteer activities, requests, and applications, helping us monitor and analyse trends effectively.

3. SQL Implementation

3.1 Database changes from 2.1



There are some changes in the UML in part 1, which are:

- Add cityID into the beneficiary
- Change interest attribute to title attribute into volunteerRequest
- Add interest relation
- Connect beneficiary relation to city relation so beneficiary will have cityID as an attributes
- Delete the importance attribute in requestSkill relation.

3.2 SQL queries

The reference results for the queries are included in their respective tables in Appendix A.

3.2.1 Query 1

This query is to include the starting and ending date in the title. This was done by concatenating the existing title with the start and end dates in the *volunteerRequest* table.

3.2.2 Query 2

This query is to find volunteers whose skill assignments match the requesting skills ordered by most matching skills to the least. This was achieved by joining the *volunteer*, *application*, *requestSkill*, and *volunteerSkill* tables.

3.2.3 Query 3

The task for this query is to show the missing number of volunteers needed per skill for each request. It does this by filtering valid applications, matching volunteer skills with requested skills, and calculating the difference between the number of volunteers needed and the number available. The results show requests with unfilled skill requirements, ordered by request ID and skill.

3.2.4 Query 4

The following query retrieves volunteer requests along with the associated beneficiary names, sorting the requests by their priority and then by the registration deadline in ascending order.

3.2.5 Query 5

The next query finds volunteer requests that match at least two of the skills and location range of volunteers. It matches volunteer skills with request skills, checks if requests are within the volunteer's specified city range, and includes requests without specific skill requirements. The results list volunteer IDs and their corresponding request IDs, ordered by volunteer and request.

3.2.6 Query 6

This query identifies volunteer requests that match the volunteers' areas of interest and are still available for registration. It normalises the request titles and volunteer interests by removing irrelevant characters and unnecessary words. Then, it matches the normalised titles with interests and retrieves the available requests ordered by volunteer and request ID.

3.2.7 Query 7

For this query the intent is to find volunteers who have applied to requests outside their specified location range. It joins volunteers' applications with their location ranges and request locations, identifying those whose *city/D* does not match the requested city. The results show the volunteers' readiness to travel, ordered by their readiness.

3.2.8 Query 8

The following query orders the skills overall from most prioritised to least on average importance value. It is achieved by a natural join between the *requestSkill* and *volunteerRequest* tables and orders the skills by the priority of the requests in descending order.

3.2.9 Query 9

This query analyses the demand for different skills by calculating the number of requests each skill is associated with and the average number of volunteers needed for those requests.

The results are sorted by the number of requests and the average demand, providing insights into the most demanded skills.. Understanding skill demand is vital for volunteer recruitment and making sure that critical skill gaps are filled as soon as possible. By monitoring the situation of which skills are in high demand, the Red Cross can focus on acquiring and keeping volunteers with those specific skills.

3.2.10 Query 10

The next query identifies the top ten volunteers who have submitted the most applications. This highlights the most active volunteers, which can be useful for recognizing and motivating them to continue their contributions. This can motivate them to continue their high level of engagement and inspire other volunteers to increase their participation. This can be done by just posting the volunteers' names on their website. Or alternatively this can be used to give a break to people who are volunteering a lot.

3.2.11 Query 11

This query is to find volunteer requests that have received fewer applications than the number of volunteers needed. This helps identify gaps in volunteer recruitment, which can be used to give incentives in order to promote these requests or change their requirements. Addressing unfulfilled requests is very

important for meeting the needs of beneficiaries and not running out of volunteers in an important task.

3.2.12 Query 12

The last query lists volunteers who have never applied to any requests. This information can be used to motivate these volunteers to participate more actively or to remove them from the system in order to not clog it up.

3.3 Views

The reference results for the views are included in their respective tables in Appendix A.

3.3.1 View 1

For this view the aim was to list the average number and age of volunteers that applied, and the average number of volunteers needed for each beneficiary. This is achieved by joining the *VolunteerRequest*, *Application*, *Volunteer*, and *Beneficiary* tables and performing calculations on them.

3.3.2 View 2

The following view provides insights into the performance of volunteers by averaging their application and skill data. It calculates the total number of applications and skills for each volunteer, as well as the average number of skills per application. This view joins the number of applications per volunteer and skills per volunteer. By calculating the total number of applications and skills for each volunteer, along with the average number of skills per application, active and skilled volunteers can be identified and their level of participation increased.

3.4 Trigger and functions

The first task was to create a check constraint with a function that validates a volunteer ID when a new volunteer is inserted. This is important, because the ID should be valid in order to recognise the volunteer, otherwise they might be lost in the system. The length should be 11 characters, the seventh character is one of “+”, “-”, “A”, “B”, “C”, “D”, “E”, “F”, “X”, “Y”, “W”, “V”, “U” and the correct control character is used.

Secondly, a trigger was created that updates the number of volunteers for a request as soon as the minimum need for any of its skill requirements is changed. The sum of unskilled volunteers and the minimum need for each required skill is used as the total number of volunteers needed for each request.

3.5 Transactions

For the first part of the transaction, the function `assign_volunteers` will filter the valid application with the skill value in the descending order. After that, it checks the register by date, and if the time is already past, the function will go to the next requestID. If it is not, it will assign the volunteer to the volunteerAssignment table.

For the second part of the transaction, the purpose of the transaction is to find any volunteers with specific skills and live in the same city as the cityID of relation requestLocation. The tasks are: filter the volunteer with valid application first and next filter any volunteers in the same city as the cityID of relation requestLocation. Moreover, it also filters the volunteers with specific skills (Communication and Marketing, Photography and Video).

3.6 Analysis

For the first part of the analysis, the number of volunteers available by city was checked. A visualisation was created that compared the number of volunteers available and the number of volunteers that applied in each city. The top 2 were Rusko and Hailuoto, indicating a high demand for volunteer services in these cities. The two lowest 2 cities were Rautavaara and Säkylä.

For the second analysis, we came up with the scoring system for matching volunteers to requests like this:

Condition: Application has to be valid

- Skill Match (40%):
 - Check if the volunteer possesses at least one skill required for the request.
 - If the requestID does not have any requestSkill, the Skill Score will automatically be 40.
 - Weight: 40%
 - Formula: $Skill\ Score = (Has\ at\ least\ 1\ matching\ skill) \times 40$
- Travel Readiness (20%):
 - The volunteer's travel readiness indicates how quickly they can be ready to travel.
 - Lower readiness score (indicating quicker readiness) will be better.
 - Weight: 20%
 - Formula: $Travel\ Readiness\ Score = (1 - \frac{Volunteer\ Readiness}{Max\ Readiness}) \times 20$
- Volunteer Range (20%):
 - Volunteer's location range matching with the request location.
 - Assign a full score if they are, zero otherwise.

- Weight: 20%
- Formula: $\text{Range Score} = (\text{Has at least 1 matching city}) \times 20$
- Interest Alignment (20%):
 - Check if the volunteer's interests match any of the skills required for the requests they have applied for.
 - If there's a match, assign a score based on the number of matching interests.
 - Weight: 20%
 - Formula: $\text{InterestScore} = \frac{\text{Number of Matching Interests}}{\text{Total Required Skills}} \times 20$

Moreover, if there is no required skill in the volunteerRequest, the skill score will be 40 automatically for that requestID.

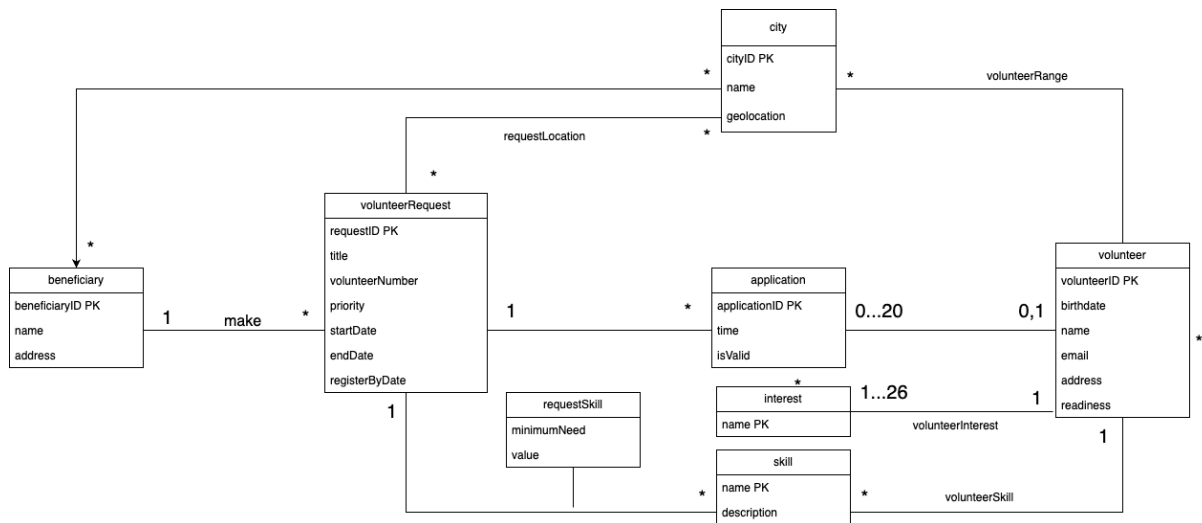
The output for this analysis is to show the top 5 candidates for each request according to this scoring system (Table 17).

For the comparison, as we can see (from table 17 and table 15), the results are not exactly the same (it matches 3 out 5 with the requestID = 1 for example). Because in the transaction part, skills account for 100% of the score. But in this part, it just accounts for 40%.

After that, the number of valid volunteer applications was compared to the number of valid requests. The months with the most applications were around summertime and the least towards the end of the year. There exists a very slight correlation between requests and applications with there being more requests during the summer as well, however the trend is much more stable.

Lastly, the number of volunteers per city and average travel readiness were compared. A slight negative correlation -0.35 was found, suggesting that more volunteers in a city means that there are less travel ready people.

Final Part



UML diagram

Modification

1. volunteerSkill, volunteerInterest, volunteerRange, requestSkill, requestLocation become association classes. So their ID won't necessary anymore (requestLocationID for example)
2. Link city to volunteerRequest
3. Link city to beneficiary
4. Link volunteerRequest to skill
5. Link city to volunteer

Relational Model

Relation Class

beneficiary (beneficiaryID, name, address)

volunteerRequest (requestID, beneficiaryID, volunteerNumber, priority, interest, startDate, endDate, registerByDate)

application (applicationID, requestID, volunteerID, time, isValid)

volunteer (volunteerID, name, birthdate, email, address, readiness)

skill (name, description)

city (cityID, name, geolocation)

view(beneficiaryID, applicationID)

Combined Class

beneficiary (beneficiaryID, name, address,cityID)

volunteerRequest (requestID, beneficiaryID, volunteerNumber, priority,
 interest, startDate, endDate, registerByDate)
 application (applicationID, requestID, volunteerID, time, isValid)
 volunteer (volunteerID, name, birthdate, email, address, readiness)
 skill (name, description)
 city (cityID, name, geolocation)
 view(beneficiaryID, applicationID)
 requestSkill (name, requestID, minimumNeed, importance,)
 requestLocation (city, requestID)
 volunteerRange (volunteerID, cityID)
 volunteerInterest (name, volunteerID)
 volunteerSkill (skill, volunteerID)

Association Class

requestSkill (name, requestID, minimumNeed, importance,)
 requestLocation (city, requestID)
 volunteerRange (volunteerID, cityID)
 volunteerInterest (name, volunteerID)
 volunteerSkill (skill, volunteerID)
 beneficiary (beneficiaryID, name, address,cityID)
 application (applicationID, requestID, volunteerID, time, isValid)

BCNF Analysis

Beneficiary

Functional Dependencies (FDs):

beneficiaryID → name, address, cityID

Analysis:

beneficiaryID is the candidate key and determinant. This table is in BCNF.

VolunteerRequest

FDs:

requestID → beneficiaryID, volunteerNumber, priority, interest, startDate, endDate,
registerByDate

Analysis:

requestID is the candidate key and determinant. This table is in BCNF.

Application

FDs:

applicationID → requestID, volunteerID, time, isValid

Analysis:

applicationID is the candidate key and determinant. This table is in BCNF.

Volunteer

FDs:

volunteerID → name, birthdate, email, address, readiness

Analysis:

volunteerID is the candidate key and determinant. This table is in BCNF.

Skill

FDs:

name → description

Analysis:

name is the candidate key and determinant. This table is in BCNF.

City

FDs:

cityID → name, geolocation

Analysis:

cityID is the candidate key and determinant. This table is in BCNF.

View

FDs:

beneficiaryID, applicationID → beneficiaryID, applicationID

Analysis:

The left-hand side is a candidate key. This table is in BCNF.

RequestSkill (Association Class)

FDs:

requestID, name $\rightarrow$ minimumNeed, importance

Analysis:

The combination of requestID and name forms a composite key, ensuring uniqueness. This table is in BCNF.

RequestLocation (Association Class)

FDs:

requestID, cityID $\rightarrow$ requestLocation

Analysis:

The combination of requestID and cityID forms a composite key, ensuring uniqueness. This table is in BCNF.

VolunteerRange (Association Class)

FDs:

volunteerID, cityID $\rightarrow$ volunteerRange

Analysis:

The combination of volunteerID and cityID forms a composite key, ensuring uniqueness. This table is in BCNF.

VolunteerInterest (Association Class)

FDs:

volunteerID, name $\rightarrow$ volunteerInterest

Analysis:

The combination of volunteerID and name forms a composite key, ensuring uniqueness. This table is in BCNF.

VolunteerSkill (Association Class)

FDs:

volunteerID, skill $\rightarrow$ volunteerSkill

Analysis:

The combination of volunteerID and skill forms a composite key, ensuring uniqueness. This table is in BCNF.

Anomaly Analysis

- Insertion Anomaly:

Example: Adding a new volunteer without specifying skills can lead to incomplete data. By treating VolunteerSkill as an association class, we ensure that both volunteerID and skill are required, preventing partial data entry.

- Update Anomaly:

Example: If a volunteer's address changes, it should be updated across all related records. Storing the volunteer's address only in the Volunteer table ensures a single point of update, avoiding inconsistencies.

- Deletion Anomaly:

Example: Deleting a volunteer should not remove critical skill data. By using VolunteerSkill as an association class, we ensure that skills are retained even if a volunteer is removed, preserving important data relationships.

- Redundancy:

Example: Redundant data occurs when the same information is stored in multiple places. By normalising tables and ensuring each piece of data is stored only once (e.g., city information in the City table), we minimise redundancy.

- Data Integrity:

Example: Maintaining foreign key constraints ensures that relationships between tables are valid. For instance, requestID in RequestSkill must exist in VolunteerRequest, maintaining referential integrity.

4. Dynamic volunteer matching

A dynamic volunteer matching system is a software solution designed to allocate volunteers to requests based on real-time data. It ensures that volunteers with the appropriate skills, availability and readiness to travel are assigned to requests. It is a valuable asset for any time, but especially during extreme situations like disasters, crises or any scenario where volunteer needs can change quickly and unpredictably.

The system was achieved by retrieving current data on volunteer requests and available volunteers. It matches volunteers to requests based on skill requirements and updates the database to reflect these assignments. To maintain the dynamic nature of the system, a scheduler was implemented to run this matching process at regular time periods in order for the system to respond to new data. The first few potential matches can be seen in Table 18.

5. Volunteer Management Dashboard

The Volunteer Management Dashboard is designed to provide an interactive and comprehensive view of volunteer activities, requests, and applications. This dashboard helps administrators and stakeholders to monitor and analyse volunteer engagement, request fulfilment, application trends, etc.

The dashboard consists of several interactive components:

- Date Range Picker: Allows users to filter data based on a selected date range.
- Request ID filter: Allows users to filter data based on selected request ID(s).
- Beneficiary filter: Allows users to filter data based on selected beneficiaries.
- Overview metrics:
 - Total Volunteers: The total number of volunteers.
 - Total Beneficiaries: The total number of beneficiaries.
 - Total Active Requests / Total Requests: The number of currently active requests (based on End Date $\geq$ Today) compared to the total number of requests.
 - Total Valid Applications / Total Applications: The number of valid applications compared to the total number of applications.
- Graphs and Charts:
 - Volunteers by City: Shows the distribution of volunteers across different cities.
 - Age Distribution of Volunteers: Provides an age-wise distribution of volunteers.
 - Requests by Priority: Displays the number of requests categorised by their priority levels.
 - Requests by City: Shows the number of requests in each city.
 - Requests and Applications Over Time: Compares the trends of requests and applications over time.
 - Skills Distribution: Visualises the distribution of skills among volunteers.
 - Interests Distribution: Illustrates the various interests of volunteers.
 - Applications per Request: Shows the number of applications received for each request title.
 - Valid and Invalid Applications: Displays a pie chart differentiating valid and invalid applications.

All the instructions for users are displayed in the README file in the zip folder.

The Volunteer Management Dashboard is a powerful tool for managing and analysing volunteer activities and requests. By offering interactive filters and comprehensive visualisations, it enables users to gain valuable insights and make informed decisions to enhance volunteer engagement and request fulfilment.

6. Roles, Responsibilities, and Reflections

For part one it meant that the group was divided into pairs: Hoang Tran and Marichka Horodyska worked on the UML implementation, while Egor Kokko and Anh Pham worked on the relational model.

For part 2 it was a little more difficult, but we decided to split the work based on the points available.

- Hoang Tran: Modified UML diagram, relational model, first-stage SQL implementation.
- Anh Pham and Egor Kokko: Basic section.
- Egor Kokko: Views.
- Anh Pham: Trigger and functions.
- Hoang Tran: Transactions.
- Anh Pham and Egor Kokko: Analysis.

Despite some issues during the working progress, we helped each other and finished the project on time.

The group work was good. One challenge that was faced were issues with connecting to the database. For example, some of the group members had trouble with data insertion, while others couldn't connect through the vpn. However, this was overcome and we got to work on the database itself. Another challenge was time pressure. We should have started working a little bit earlier than we did, but we managed and it will be something to take into account for the next group project.

For the final deliverable, we splitted the workload to 2 parts:

- Egor Kokko and Marichka Horodyska: Dynamic Volunteer Matching.
- Anh Pham and Hoang Tran: Volunteer Management Dashboard.

This was a very interesting course that will definitely help us in the future. There were many useful learning outcomes. For one we learned how to design a database from scratch (the UML, relational model, data cleaning, data insertion among others). We learned how to use python (pandas) to connect to databases, how to enforce data integrity through constraints, triggers and validation rules and how to implement a transaction.

7. Conclusion

The project is to design a Volunteer Matching System (VMS) in collaboration with the Finnish Red Cross (FRC). Our primary goal was to create a system that efficiently connects volunteers with beneficiary requests, ensuring that the right skills and resources are allocated where they are most needed.

We began by constructing a detailed UML diagram and relational model, which formed the foundation of our database structure. This initial phase was crucial in defining the relationships and attributes necessary for the VMS. Moving forward, the SQL implementation phase allowed us to translate these models into practical components. We developed a series of SQL queries, views, triggers, and functions to support the system's operations, ensuring data integrity and automating key processes.

One of the achievements of this project was the dynamic volunteer matching mechanism, which enables real-time allocation of volunteers based on current data. This feature is useful during emergencies, where volunteer needs can change rapidly. Moreover, the volunteer Management Dashboard we created provides an interactive and comprehensive view of volunteer activities, total requests, beneficiaries, and volunteers.

Through the project and the course, we faced some challenges, from technical difficulties with database connectivity to time management issues. Thanks to that, we have chances to find the solutions and learn more.

In conclusion, the Volunteer Matching System we developed is a practical tool that allows the Finnish Red Cross to enhance their ability to manage their volunteer resources effectively.

7. Appendix A

(This appendix is for explanation to some questions. Every other outputs / answers will be shown in the ipynb file)

name	volunteerid	requestid	skill_match
Mikko Rossi	230283-963X	1	3
Henna Hartikainen	011074-9149	1	1
Matilda Tuominen	211074-9401	1	1
Anniina Saastamoinen	211099-910H	1	1
Johannes Jäntti	160903A941P	1	1
...	...	...	...
Juhani Pesonen	010573-901K	382	7
Olavi Karvinen-Reinikainen	231269-913S	382	5
Outi Luukkonen-Timonen	100396-906V	382	3
Pekka Peltonen-Koivisto	200472-937X	382	1
Karoliina Kallio	210694-902R	382	0

Table 2. Query 2

requestid	request_skill	minimumneed	volunteer_count	missing_volunteers
1	CommunicationAndMarketing	5	3	2
1	EventOrganization	2	1	1
2	CookingAndBaking	3	1	2
2	PublicPerformances	3	2	1
2	TeamGuide	4	2	2
...	...	...	...	...
382	DigitalCompetence	3	1	2
382	EventHosting	2	1	1
382	Organizational	4	2	2
382	PhotographyAndVideo	3	1	2
382	TrainPeople	3	2	1

Table 3. Query 3

requestid	title	priority	registerbydate	beneficiary_name
369	guide and teach needed	5	2019-12-26	Youth Centre
127	food help needed	5	2020-01-03	Youth Centre
92	collect donations needed	5	2020-01-29	Food Bank
133	promote wellbeing needed	5	2020-03-07	Elderly Care
224	collect donations needed	5	2020-09-11	Immigration
...	...	...	...	...
202	immigrant support needed	0	2024-11-08	Blood-Drive (PA)
24	food help needed	0	2024-11-12	Youth Centre
301	guide and teach needed	0	2024-11-23	Event First-Aid
81	collect donations needed	0	2024-11-26	Blood-Drive (PA)
355	help in crisis needed	0	2024-12-02	Event First-Aid

Table 4. Query 4

volunteerid	requestid
010469-981A	1
010469-981A	2
010469-981A	5
010469-981A	6
010469-981A	10
...	...
311267-9044	370
311267-9044	372
311267-9044	379
311267-9044	381
311267-9044	382

Table 5. Query 5

volunteerid	requestid	title
010469-981A	24	food help needed
010469-981A	88	food help needed
010469-981A	220	food help needed
010469-981A	236	organise activities needed
010469-981A	241	organise activities needed
...	...	...
311064-910X	351	collect donations needed
311064-910X	356	collect donations needed
311064-910X	358	organise activities needed
311064-910X	376	collect donations needed
311064-910X	377	collect donations needed

Table 6. Query 6

requestid	volunteerid	name	email	readiness
82	111276-945V	Kalervo Rinne	kalervo.rinne@dbcourse.cs.aalto.fi	1439
146	111276-945V	Kalervo Rinne	kalervo.rinne@dbcourse.cs.aalto.fi	1439
146	111276-945V	Kalervo Rinne	kalervo.rinne@dbcourse.cs.aalto.fi	1439
227	111276-945V	Kalervo Rinne	kalervo.rinne@dbcourse.cs.aalto.fi	1439
227	111276-945V	Kalervo Rinne	kalervo.rinne@dbcourse.cs.aalto.fi	1439
...	...	...	...	...
14	230168-9007	Maria Hietanen-Parviainen	maria.hietanen-parviainen@dbcourse.cs.aalto.fi	64
14	230168-9007	Maria Hietanen-Parviainen	maria.hietanen-parviainen@dbcourse.cs.aalto.fi	64
14	230168-9007	Maria Hietanen-Parviainen	maria.hietanen-parviainen@dbcourse.cs.aalto.fi	64
148	230168-9007	Maria Hietanen-Parviainen	maria.hietanen-parviainen@dbcourse.cs.aalto.fi	64
148	230168-9007	Maria Hietanen-Parviainen	maria.hietanen-parviainen@dbcourse.cs.aalto.fi	64

Table 7. Query 7

name	priority
Organizational	5
CommunicationAndMarketing	5
MeetingPeople	5
DigitalCompetence	5
EventHosting	5
...	...
EventOrganization	0
PhotographyAndVideo	0
CookingAndBaking	0
EventHosting	0
Organizational	0

Table 8. Query 8

skill_name	num_requests	avg_skill_demand
EventOrganization	151	3.37
PhotographyAndVideo	148	3.61
FinanceAndAccounting	148	3.40
CookingAndBaking	148	3.30
Organizational	143	3.41
DigitalCompetence	143	3.38
Rescue	142	3.40
HealthCareOrFirstAid	136	3.63
CommunicationAndMarketing	135	3.45
EventHosting	135	3.44
TrainPeople	132	3.52
MeetingPeople	128	3.65
PublicPerformances	128	3.57
TeamGuide	124	3.77

Table 9. Query 9

volunteer_id	volunteer_name	number_of_applications
170254-9461	Elina Kärki	30
210489-9136	Kauko Holm	29
220857-9810	Ville Honkanen	29
231259-9690	Mika Martikainen	28
220682-954Y	Helena Salminen	28
011074-9149	Henna Hartikainen	28
100766-9636	Juhani Järvenpää	28
220782-910B	Pauliina Männistö	28
230881-9561	Annikki Savolainen	28
210753-990T	Oona Kauppinen	27

Table 10. Query 10

requestid	request_name	number_needed	number_of_applications
1	work in team needed	14	7
2	work with young needed	25	6
3	guide and teach needed	31	5
4	guide and teach needed	19	5
5	work with elderly needed	23	7
...	...	...	...
378	work with young needed	11	7
379	immigrant support needed	28	10
380	work with young needed	18	10
381	work with young needed	20	11
382	collect donations needed	22	7

Table 11. Query 11

volunteer_id	volunteer_name	contact_info
050677-9718	Mika Moilanen	mika.moilanen@dbcourse.cs.aalto.fi
250162-950K	Helinä Kukkonen-Mäki	helina.kukkonen-maki@dbcourse.cs.aalto.fi
250672-987C	Aimo Vuorela	aimo.vuorela@dbcourse.cs.aalto.fi
280104A9662	Teija Turpeinen-Matikainen	teija.turpeinen-matikainen@dbcourse.cs.aalto.fi
280190-969B	Petteri Heinonen-Jääskeläinen	petteri.heinonen-jaaskelainen@dbcourse.cs.aalto.fi

Table 12. Query 12

beneficiaryid	beneficiary_name	volunteers_applied	age_volunteers	volunteers_needed
1	Hospital	253	42.89	20.65
2	Food Bank	327	45.8	18.98
3	Elderly Care	250	43.75	17.76
4	Youth Centre	323	44.83	17.77
5	Homeless Shelter	326	44.85	17.4
6	Blood-Drive (PA)	288	44.79	16.68
7	Event First-Aid	350	43.31	19.06
8	Immigration	353	44.67	19.45
9	Local Branch	254	46.54	21.08

Table 13. View 1

volunteerid	volunteer_name	application_count	skill_count	avg_skills
281170-9344	Eija Turpeinen	8	9	1.13
020798-936D	Katariina Nieminen-Nevalainen	15	7	0.47
020657-903Y	Rainer Mäkelä	18	7	0.39
231259-9690	Mika Martikainen	28	6	0.21
141199-937N	Esko Mattila	23	4	0.17
080787-976R	Irene Liimatainen	10	7	0.70
100174-937K	Robert Laine	15	12	0.80
250162-950K	Helinä Kukkonen-Mäki	0	13	0.00
190656-9517	Juha Leinonen	14	5	0.36
220559-937N	Väinö Kukkonen	13	7	0.54
...	...	...	...	...

Table 14. View 2

123 assignmentid ▼	123 requestid ▼	ABC volunteerid ▼	☒ isaccepted ▼
143	1	230283-963X	[v]
144	1	211074-9401	[v]
145	1	011074-9149	[v]
146	1	211099-910H	[v]
147	1	230283-963X	[v]
148	1	230283-963X	[v]
149	1	160903A941P	[v]
150	1	250681-919H	[v]
151	1	210753-990T	[]
152	3	030693-935X	[v]
153	3	190697-999B	[v]
154	3	100494-989U	[v]
155	3	200958-9326	[v]
156	3	030693-935X	[v]
157	3	190697-999B	[v]
158	3	200958-9326	[v]
159	3	030693-935X	[v]
160	3	200958-9326	[v]
161	3	030693-935X	[v]
162	3	190697-999B	[v]
163	3	030693-935X	[v]
164	3	190697-999B	[v]
165	3	220782-910B	[v]
166	3	100494-989U	[v]
167	3	030693-935X	[v]

Table 15. Transaction with filtering the volunteers having the matching skill set as the request.

172	3	220782-910B	[v]
173	3	100494-989U	[v]
174	3	030693-935X	[v]
175	3	190697-999B	[v]
176	3	100494-989U	[v]
177	3	200958-9326	[v]
178	3	030693-935X	[v]
179	3	190697-999B	[v]
180	3	200958-9326	[v]
181	3	030693-935X	[v]
182	3	200958-9326	[v]
183	3	030693-935X	[v]
184	3	190697-999B	[v]
185	3	030693-935X	[v]
186	3	190697-999B	[v]
187	3	220782-910B	[v]
188	3	100494-989U	[v]
189	3	030693-935X	[v]
190	3	190697-999B	[v]
191	3	220782-910B	[v]
192	3	100494-989U	[v]
193	3	190697-999B	[v]
194	3	220782-910B	[v]
195	3	100494-989U	[v]

Table 16. Transaction with filtering the volunteers having specific skills and have the same cityID as that in request.

	requestid	volunteerid	volunteer_name	score
709	1	250681-919H	Tapio Rantala	77.970813
307	1	011074-9149	Henna Hartikainen	73.634468
2121	1	211074-9401	Matilda Tuominen	73.231411
246	1	160903A941P	Johannes Jäntti	65.323141
527	1	211099-910H	Anniina Saastamoinen	62.251564
...	...	...	...	...
2269	382	100396-906V	Outi Luukkonen-Timonen	69.214732
1542	382	200472-937X	Pekka Peltonen-Koivisto	63.432940
646	382	010573-901K	Juhani Pesonen	50.173732
2068	382	231269-913S	Olavi Karvinen-Reinikainen	46.059764
1803	382	210694-902R	Karoliina Kallio	20.138985

[1752 rows x 4 columns]

Table 17. Analysis with Scoring System

matchid	volunteerid	requestid	matchdate
1	101156-956L	1	2024-06-11
2	040375-945U	1	2024-06-11
3	270690-958C	1	2024-06-11
4	101156-956L	1	2024-06-11
5	211259-994H	1	2024-06-11
...	...	...	...
116	121095-972N	21	2024-06-11
117	040375-945U	21	2024-06-11
118	121095-972N	21	2024-06-11
119	121095-972N	21	2024-06-11
120	101156-956L	21	2024-06-11

Table 18. Dynamic matching implementation

